

Integração Contínua

Prof. Breno de França

Prof. Bruno Cafeo

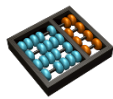
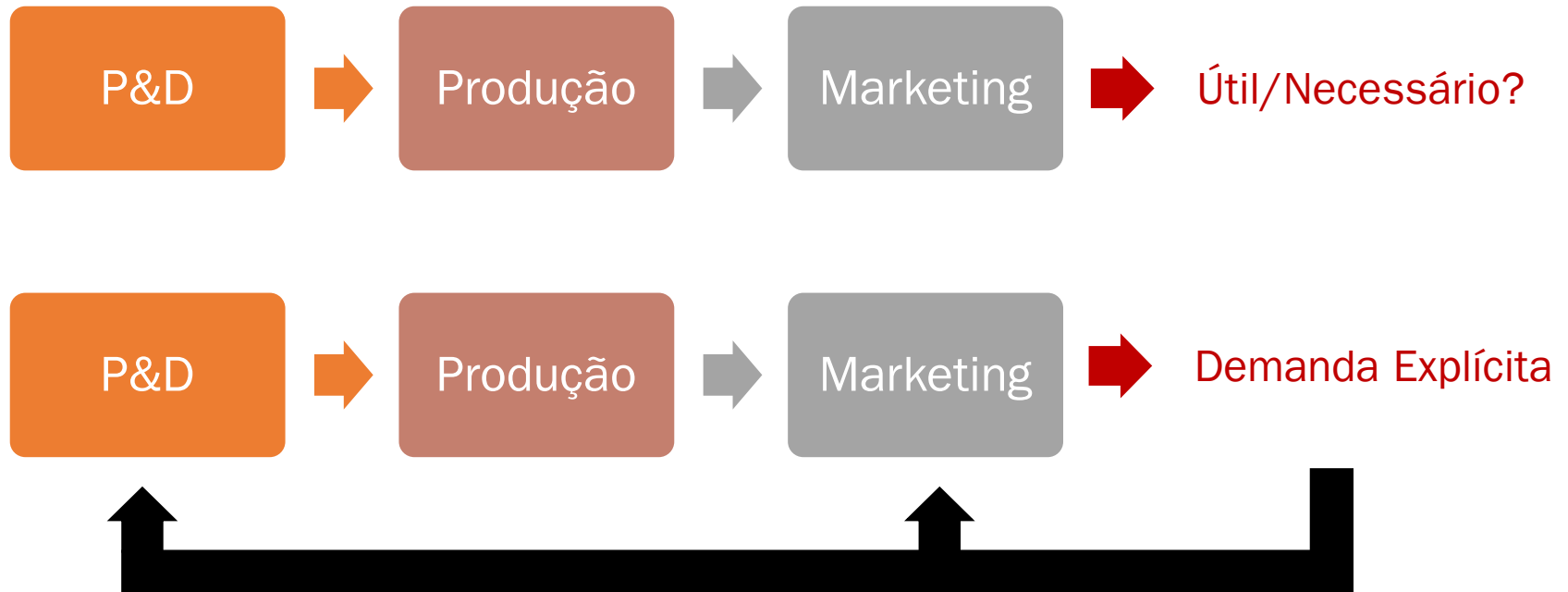
cafeo@unicamp.br

Motivações

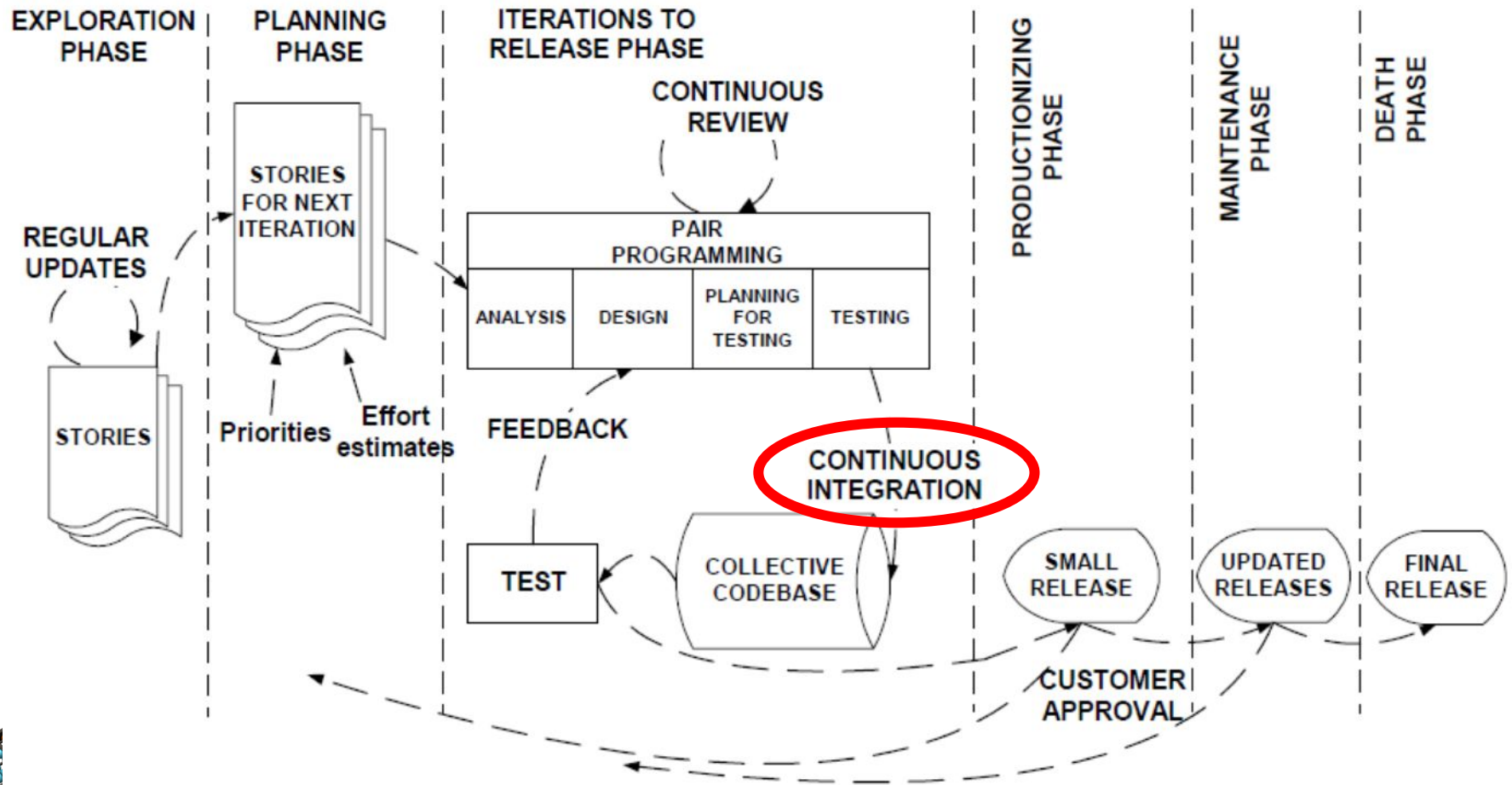
- *Time-to-Market*: Necessidade de entregas mais frequentes
 - Mudanças externas
- Demandas por qualidade
 - Requisitos não-funcionais
 - Infraestrutura complexa
- SaaS: Software as a Service 
- Processo de Liberação: efeitos inesperados
- Pouco *feedback* do uso das aplicações
- ...

Fluxo Contínuo

PUSH x PULL



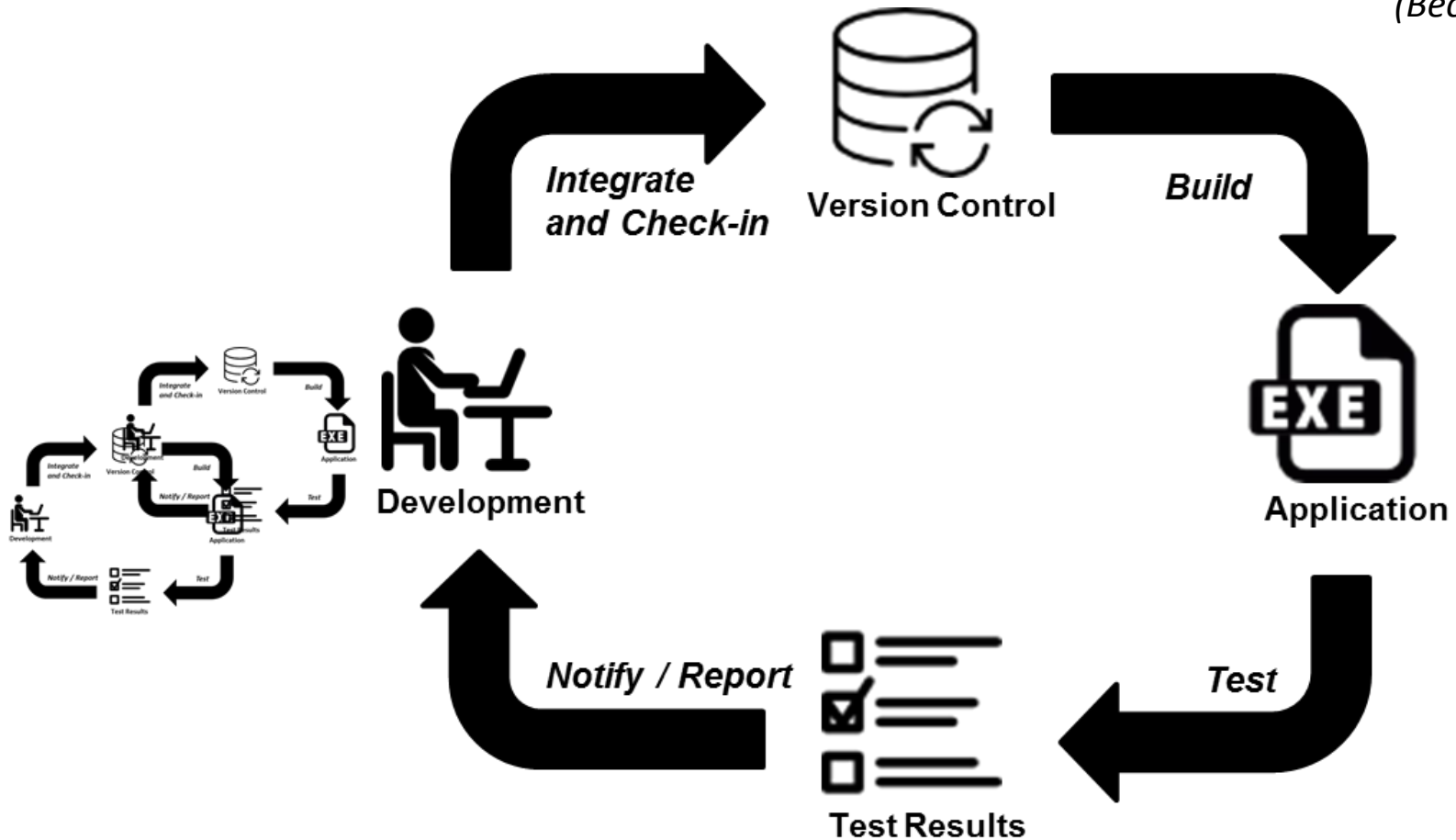
eXtreme Programming (XP)



Fonte: Abrahamsson, Pekka, et al. "Agile software development methods: Review and analysis." (2002).

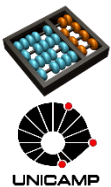
Integração Contínua

“Integrate and build the system many times a day, every time a task is completed.”
(Beck, 1999)



Integração Contínua

- Incrementos pequenos facilitam:
 - **Integração**, caso contrário:
 - Aumenta a quantidade de conflitos e o esforço de integração
 - Identificação e remoção de **defeitos**
- Aplicação sempre em estado funcional
- *Código finalizado deve ser integrado!*
 1. *Antes de integrar, checkout!*
 2. *Teste de unidade*
 3. *Após integrar, mais testes!*



+ Cobertura → + Confiança no Sistema

- Quanto do código é testado? Quanto deveria ser?

Integração Contínua

Está pronto?



Version Control

Está disponível?



Application

Executa?

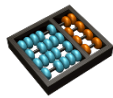


Esta Foto de Autor Desconhecido está licenciado em [CC BY-NC](#)



Test Results

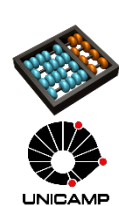
Passou nos testes com sucesso?



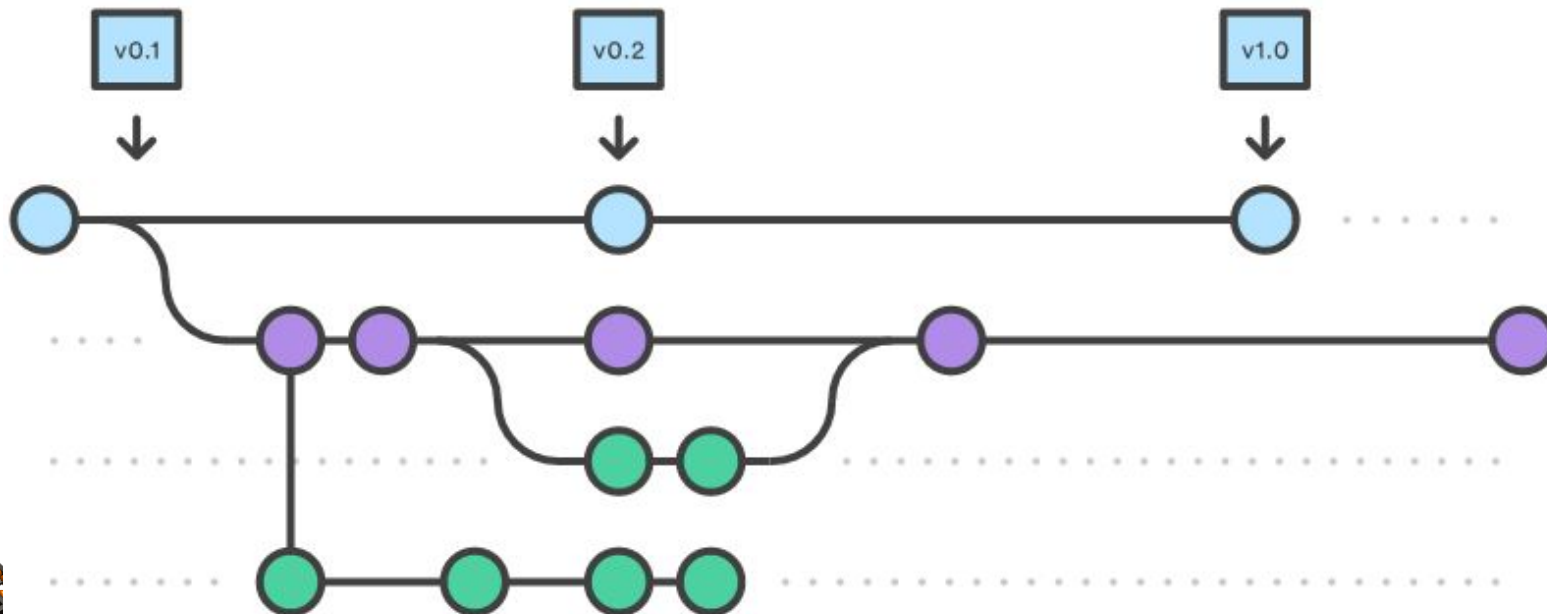
Integração Contínua

Pressupostos

1. Controle de Versão Implantado
 - Tudo necessário para instalar, executar e testar a aplicação deve estar no repositório
 - Código, configs, scripts de teste, scripts de *build* e *deployment*
2. Build Automático
 - Scripts tratado como o código da aplicação
3. Comprometimento da Equipe
 - Prática x Ferramenta



Controle de Versão



Tutorial (Youtube): https://youtu.be/SWYqp7iY_Tc
Referência Oficial : <https://git-scm.com/docs>

Build Automatizado

- Construção da aplicação por meio de scripts/ferramentas que **automatizam** o processo para gerar um “executável” da aplicação
- **Todo e qualquer arquivo necessário** para construção deve estar **disponível** em um repositório

Ferramentas:



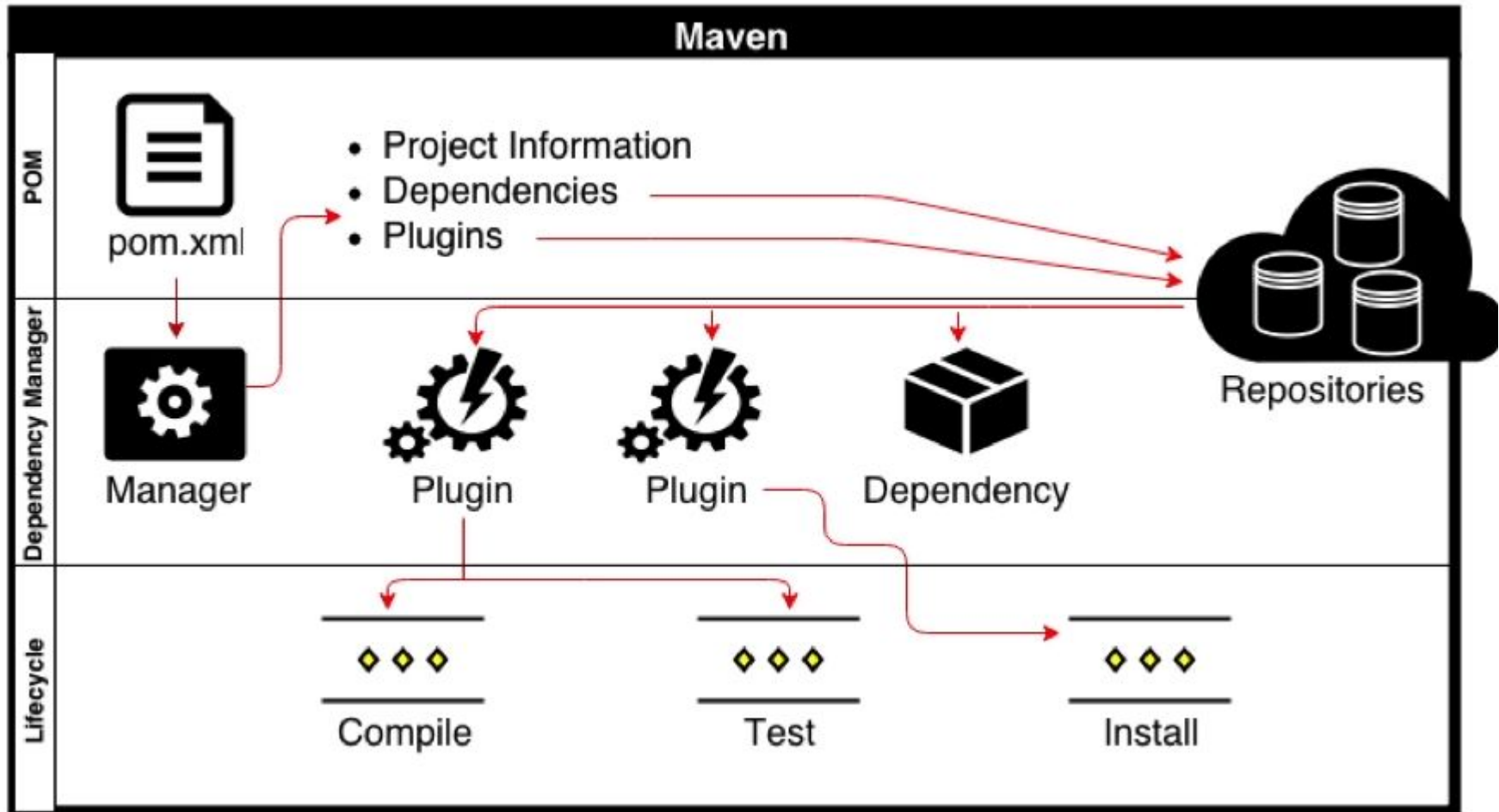
Build Automatizado com Maven

```
1. my-app
2. |-- pom.xml
3. `-- src
4.     |-- main
5.         |-- java
6.             |-- com
7.                 |-- mycompany
8.                     |-- app
9.                         |-- App.java
10.    `-- test
11.        |-- java
12.            |-- com
13.                |-- mycompany
14.                    |-- app
15.                        |-- AppTest.java
```

Aplicação

Testes

Build Automatizado com Maven



Build Automatizado com Maven

Ciclo de Vida Maven (mais comum/default)

validate

compile

test

package

integration-test

verify

install

deploy

compiler:compile

surefire:test

ejb:ejb or ejb3:ejb3 or jar:jar or par:par or rar:rar or war:war

Install:install

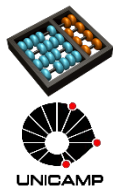
deploy:deploy



Build Automatizado com Maven

Ciclo de Vida Maven (mais comum/default)

- **validate**: validate the project is correct and all necessary information is available
- **compile**: compile the source code of the project
- **test**: test the compiled source code using a suitable unit testing framework. These tests should not require the code be packaged or deployed
- **package**: take the compiled code and package it in its distributable format, such as a JAR.
- **integration-test**: process and deploy the package if necessary into an environment where integration tests can be run
- **verify**: run any checks to verify the package is valid and meets quality criteria
- **install**: install the package into the local repository, for use as a dependency in other projects locally
- **deploy**: done in an integration or release environment, copies the final package to the remote repository for sharing with other developers and projects.
- There is other Maven lifecycles , but to of note beyond the *default* list above:
- **clean**: cleans up artifacts created by prior builds

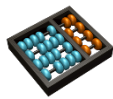


Servidores de IC

- Funcionam como “escalonadores de tarefas”
- Normalmente, seguem um fluxo padrão de IC com alguma flexibilidade
- Fortemente baseados em plugins e integrações
- Exemplos:



Travis CI



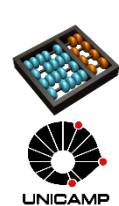
Integração Contínua no GitLab



- GitLab Runner é um serviço integrado com base no Travis CI
- Para cada projeto é possível associar distintos Runners
- Para usar e configurar, basta adicionar um arquivo YAML na raiz do projeto:

- .gitlab-ci.yml

```
image: ruby:2.1
services:
  - postgres
before_script:
  - bundle install
after_script:
  - rm secrets
stages:
  - build
  - test
  - deploy
job1:
  stage: build
  script:
    - execute-script-for-job1
  only:
    - master
tags: - docker
```

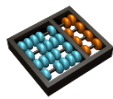


Integração Contínua no GitLab



.gitlab-ci.yml

Keyword	Description
image	Use docker image, covered in Use Docker
services	Use docker services, covered in Use Docker
stages	Define build stages
before_script	Define commands that run before each job's script
after_script	Define commands that run after each job's script
variables	Define build variables
cache	Define list of files that should be cached between subsequent runs

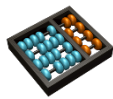


Integração Contínua no GitHub



`.github/workflows/*.yml`

Keyword	Description
name	nome do workflow
run_name	nome de uma execução, normalmente parametrizado
on	eventos que disparam o workflow
permissions	permissões (quem pode disparar o workflow)
env	configurações de ambiente
jobs	tarefas a serem realizadas



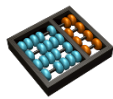
Consultar sintaxe em: <https://docs.github.com/en/actions/using-workflows/workflow-syntax-for-github-actions>



Integração Contínua

Antes de submeter uma nova versão:

1. Há uma construção (*build*) em andamento?
 - Sim? Espere terminar
 - Se falhar, espere a correção ou voltar a versão antes de submeter uma nova modificação
2. Uma vez construído e testes realizados com sucesso, sincronize o código no seu *workspace* com a última versão do repositório
3. Execute os scripts de *build* e teste localmente
4. Se tudo bem, submeta as alterações para o repositório
5. Espere o servidor de IC construir suas novas alterações
6. Falhou? Pare e corrija imediatamente no seu *workspace* — Volte ao passo 3
7. Se passar, “comemore” e pode iniciar a próxima issue

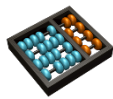


Humble, Jez, and David Farley. *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation* (Adobe Reader). Pearson Education, 2010.

Integração Contínua

Práticas e recomendações

- Submeta novas versões (*check in*) regularmente
- Crie uma suíte de testes automatizados com boa cobertura
- Mantenha os processos de *build* e teste em curta duração
 - Gargalos para novos *check in*
 - Difícil identificação e resolução de problemas
- Não faça *check in* se o *build* (local e remoto) não estiver funcionando
- Sempre execute testes antes de submeter a nova versão
 - Servidor de IC não é ferramenta de *debug*!
- Não abandone a o repositório em um estado instável
 - Volte a versão se necessário....
- Não ignore os testes que não passarem. Corrija!



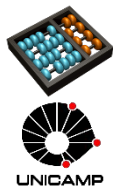
Humble, Jez, and David Farley. *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation (Adobe Reader)*. Pearson Education, 2010.

Integração Contínua

Aspectos Culturais

“Commit Daily, Commit Often” → “Build Early, Build Often”

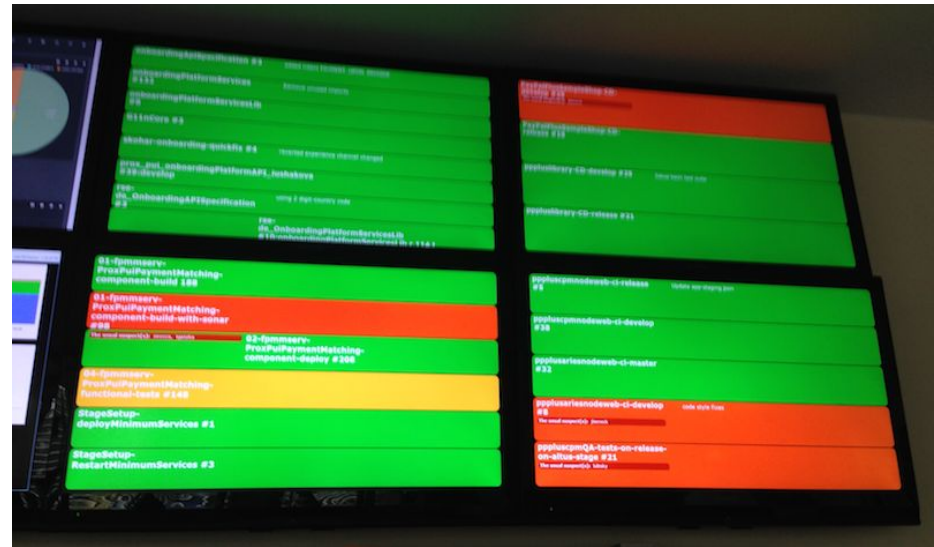
- Ferramentas não bastam, é preciso mudar a forma de pensar em como o software é desenvolvido
- Integrar com frequência apresenta benefícios, mas precisa disciplina



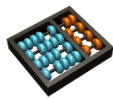
Integração Contínua

Aspectos Culturais

- “*Keep It Green*” → Awareness!
 - Medição e monitoração das modificações



- Comunicação é essencial!
 - Mudanças críticas sempre requerem **avaliação** e **comunicação** com outros membros da equipe



Build Monitors

